Referencje i kopie — zagnieżdżone struktury obiektów

Wyobraźmy sobie serwis, w którym użytkownicy zapisują ulubione trasy spacerowe, biegowe lub rowerowe. Trasa ma nazwę i przebieg opisany listą punktów. Użytkownik może skopiować ulubione trasy innej osoby, a następnie dostosować je do swoich potrzeb. Chcemy, żeby późniejsze zmiany jego kopii nie wpływały na trasy źródłowe.

Zaczniemy od zbudowania takiej struktury. Następnie sprawdzimy, które jej elementy są wspólne, i porównamy przypisanie, płytką kopię i głęboką kopię. Definicje i kolejne operacje są w osobnych komórkach, żeby można było wykonywać je krok po kroku.

Punkty, trasy i użytkownicy

Klasa `Wspolrzedne` przechowuje długość i szerokość geograficzną punktu.

In [1]:
```
class Wspolrzedne:
    def __init__(self, dlugosc, szerokosc):
        self.dlugosc = dlugosc
        self.szerokosc = szerokosc

    def __repr__(self):
        return f"Wspolrzedne({self.dlugosc!r}, {self.szerokosc!r})"
```

Reprezentacja diagnostyczna

Metoda `__repr__` powinna, jeśli to możliwe, zwracać wyrażenie pozwalające odtworzyć obiekt o tej samej wartości. Jeśli nie jest to możliwe, dokumentacja zaleca opis w nawiasach ostrych `<...>`.

Na przykład obiekt pliku zwrócony przez `open` może mieć taką reprezentację (przykładowy wynik):

`<_io.TextIOWrapper name='dane.csv' mode='r' encoding='utf-8'>`
Nie jest to wywołanie konstruktora, ale opis zawierający nazwę pliku, tryb otwarcia i kodowanie.

W naszych klasach `Trasa` i `Uzytkownik` konstruktory nie przyjmują całego stanu, który chcemy pokazać. Dlatego ich reprezentacje zapiszemy jako opisy w nawiasach ostrych.

Trasa przechowuje nazwę i listę punktów. Metoda `dodaj_punkt` dopisuje otrzymany punkt do listy — nie kopiuje go.

In [2]:
```
class Trasa:
    def __init__(self, nazwa):
        self.nazwa = nazwa
        self.przebieg = []

    def dodaj_punkt(self, punkt):
        self.przebieg.append(punkt)

    def __repr__(self):
        return f"<Trasa nazwa={self.nazwa!r}, przebieg={self.przebieg!r}>"
```

Użytkownik ma listę ulubionych tras. Na razie zdefiniujemy tylko dodawanie trasy. Również ta operacja nie tworzy kopii.

Metody `__repr__` pokazują stan zagnieżdżonych obiektów, żeby łatwiej było go obserwować. Dla `Trasa` i `Uzytkownik` są to opisy diagnostyczne w nawiasach ostrych.

In [3]:
```
class Uzytkownik:
    def __init__(self, imie, nazwisko):
        self.imie = imie
        self.nazwisko = nazwisko
        self.ulubione_trasy = []

    def dodaj_ulubiona(self, trasa):
        self.ulubione_trasy.append(trasa)

    def __repr__(self):
        return (
            f"<Uzytkownik imie={self.imie!r}, "
            f"nazwisko={self.nazwisko!r}, "
            f"ulubione_trasy={self.ulubione_trasy!r}>"
        )
```

In [4]:
```
dom = Wspolrzedne(22.55, 51.24)
praca = Wspolrzedne(22.57, 51.25)
```

In [5]:
```
trasa = Trasa("Z domu do pracy")
```

In [6]:
```
trasa.dodaj_punkt(dom)
trasa.dodaj_punkt(praca)
```

In [7]:
```
trasa
```

Out[7]:
```
<Trasa nazwa='Z domu do pracy', przebieg=[Wspolrzedne(22.55, 51.24), Wspolr
zedne(22.57, 51.25)]>
```

In [8]:
```
jan = Uzytkownik("Jan", "Kowalski")
```

In [9]:
```
jan.dodaj_ulubiona(trasa)
```

In [10]:
```
jan
```

Out[10]:
```
<Uzytkownik imie='Jan', nazwisko='Kowalski', ulubione_trasy=[<Trasa nazwa
='Z domu do pracy', przebieg=[Wspolrzedne(22.55, 51.24), Wspolrzedne(22.57,
51.25)]>]>
```

Dostęp do zagnieżdżonych danych

Możemy przechodzić od użytkownika do trasy, od trasy do punktu, a następnie do pola tego punktu:

In [11]:
```
jan.ulubione_trasy[0]
```

Out[11]:
```
<Trasa nazwa='Z domu do pracy', przebieg=[Wspolrzedne(22.55, 51.24), Wspolr
zedne(22.57, 51.25)]>
```

In [12]:
```
jan.ulubione_trasy[0].przebieg
```

Out[12]:
```
[Wspolrzedne(22.55, 51.24), Wspolrzedne(22.57, 51.25)]
```

In [13]:
```
jan.ulubione_trasy[0].przebieg[1]
```

Out[13]:
```
Wspolrzedne(22.57, 51.25)
```

In [14]:
```
jan.ulubione_trasy[0].przebieg[1].dlugosc
```

Out[14]:
```
22.57
```

To nie są kopie. Sprawdźmy, że na liście znajduje się ten sam obiekt punktu, który przekazaliśmy do metody:

In [15]:
```
jan.ulubione_trasy[0].przebieg[0] is dom
```

Out[15]:
```
True
```

In [16]:
```
jan.ulubione_trasy[0].przebieg[0].dlugosc = 22.56
```

In [17]:
```
dom
```

Out[17]:
```
Wspolrzedne(22.56, 51.24)
```

Możemy też użyć tego samego punktu na innej trasie. Zmiana jego współrzędnych będzie wtedy widoczna na obu trasach.

In [18]:
```
anna = Uzytkownik("Anna", "Nowak")
dom_anny = Wspolrzedne(22.60, 51.26)
trasa_anny = Trasa("Z domu Anny do pracy")
```

In [19]:
```
trasa_anny.dodaj_punkt(dom_anny)
trasa_anny.dodaj_punkt(praca)
anna.dodaj_ulubiona(trasa_anny)
```

In [20]:
```
jan.ulubione_trasy[0].przebieg[1].dlugosc = 22.58
```

In [21]:
```
anna.ulubione_trasy[0].przebieg[1]
```

Out[21]:
```
Wspolrzedne(22.58, 51.25)
```

Dlaczego samo przypisanie nie wystarcza?

Anna chce teraz skopiować ulubione trasy Jana. Utworzymy ją ponownie, żeby zacząć z pustą listą ulubionych.

In [22]:
```
anna = Uzytkownik("Anna", "Nowak")
```

In [23]:
```
anna.ulubione_trasy = jan.ulubione_trasy
```

In [24]:
```
anna.ulubione_trasy is jan.ulubione_trasy
```

Out[24]:
```
True
```

In [25]:
```
anna.dodaj_ulubiona(Trasa("Spacer Anny"))
```

In [26]:
```
jan.ulubione_trasy
```

Out[26]:
```
[<Trasa nazwa='Z domu do pracy', przebieg=[Wspolrzedne(22.56, 51.24), Wspol
rzedne(22.58, 51.25)]>,
 <Trasa nazwa='Spacer Anny', przebieg=[]>]
```

Samo przypisanie spowodowało współdzielenie listy. Dodanie trasy przez Annę zmieniło również listę widoczną u Jana. Przywróćmy osobną listę Jana z jego pierwotną trasą przed następną próbą.

In [27]:
```
jan.ulubione_trasy = [trasa]
anna = Uzytkownik("Anna", "Nowak")
```

Płytka kopia listy

Skopiujemy listę metodą `copy`. Powstanie nowa lista, ale jej elementami będą te same obiekty tras.

In [28]:
```
anna.ulubione_trasy = jan.ulubione_trasy.copy()
```

In [29]:
```
anna.ulubione_trasy is jan.ulubione_trasy
```

Out[29]:
```
False
```

In [30]:
```
anna.dodaj_ulubiona(Trasa("Spacer Anny"))
```

In [31]:
```
len(jan.ulubione_trasy), len(anna.ulubione_trasy)
```

Out[31]:
```
(1, 2)
```

Dodawanie tras jest już niezależne. Spróbujmy jednak zmienić nazwę skopiowanej trasy:

In [32]:
```
anna.ulubione_trasy[0].nazwa = "Droga Anny do pracy"
```

In [33]:
```
jan.ulubione_trasy[0].nazwa
```

Out[33]:
```
'Droga Anny do pracy'
```

In [34]:
```
anna.ulubione_trasy[0] is jan.ulubione_trasy[0]
```

Out[34]:
```
True
```

Płytka kopia samej trasy

Skopiowanie samej trasy również nie rozwiązuje całego problemu. Powstaje nowy obiekt `Trasa`, ale jego pole `przebieg` wskazuje tę samą listę punktów.

In [35]:
```
import copy
```

In [36]:
```
kopia_trasy = copy.copy(trasa)
```

In [37]:
```
kopia_trasy is trasa
```

Out[37]:
```
False
```

In [38]:
```
kopia_trasy.przebieg is trasa.przebieg
```

Out[38]:
```
True
```

Nawet osobne skopiowanie listy punktów pozostawia wspólne obiekty współrzędnych:

In [39]:
```
kopia_trasy.przebieg = trasa.przebieg.copy()
```

In [40]:
```
kopia_trasy.przebieg is trasa.przebieg
```

Out[40]:
```
False
```

In [41]:
```
kopia_trasy.przebieg[0] is trasa.przebieg[0]
```

Out[41]: True

Głęboka kopia tras

Potrzebujemy osobnych kopii tras, ich list przebiegu i punktów. Użyjemy `deepcopy`.

Dodajmy metodę `kopiuj_ulubione`. Dopisuje ona kopie do istniejącej listy użytkownika, zamiast ją zastępować. Nie usuwa duplikatów — ponowne wywołanie dopisze kolejne kopie.

Poniższa komórka zawiera całą nową definicję klasy. Po jej wykonaniu utworzymy nowych użytkowników, żeby korzystali z tej definicji.

In [42]:
```
class Uzytkownik:
    def __init__(self, imie, nazwisko):
        self.imie = imie
        self.nazwisko = nazwisko
        self.ulubione_trasy = []

    def dodaj_ulubiona(self, trasa):
        self.ulubione_trasy.append(trasa)

    def __repr__(self):
        return (
            f"<Uzytkownik imie={self.imie!r}, "
            f"nazwisko={self.nazwisko!r}, "
            f"ulubione_trasy={self.ulubione_trasy!r}>"
        )

    def kopiuj_ulubione(self, inny):
        self.ulubione_trasy.extend(copy.deepcopy(inny.ulubione_trasy))
```

In [43]:
```
dom = Wspolrzedne(22.55, 51.24)
praca = Wspolrzedne(22.57, 51.25)
trasa = Trasa("Z domu do pracy")
trasa.dodaj_punkt(dom)
trasa.dodaj_punkt(praca)
```

In [44]:
```
jan = Uzytkownik("Jan", "Kowalski")
jan.dodaj_ulubiona(trasa)
anna = Uzytkownik("Anna", "Nowak")
```

In [45]:
```
anna.dodaj_ulubiona(Trasa("Spacer Anny"))
```

In [46]:
```
anna.kopiuj_ulubione(jan)
```

In [47]:
```
anna.ulubione_trasy
```

Out[47]:
```
[<Trasa nazwa='Spacer Anny', przebieg=[]>,
 <Trasa nazwa='Z domu do pracy', przebieg=[Wspolrzedne(22.55, 51.24), Wspol
rzedne(22.57, 51.25)]>]
```

Zmieńmy kolejno nazwę skopiowanej trasy, współrzędne jednego punktu i liczbę punktów. Po każdej operacji sprawdźmy oryginał.

In [48]:
```
kopia = anna.ulubione_trasy[1]
```

In [49]:
```
kopia.nazwa = "Droga Anny do pracy"
```

In [50]:
```
jan.ulubione_trasy[0].nazwa
```

Out[50]:
```
'Z domu do pracy'
```

In [51]:
```
kopia.przebieg[0].dlugosc = 22.60
```

In [52]:
```
jan.ulubione_trasy[0].przebieg[0]
```

Out[52]:
```
Wspolrzedne(22.55, 51.24)
```

In [53]:
```
kopia.dodaj_punkt(Wspolrzedne(22.61, 51.26))
```

In [54]:
```
len(jan.ulubione_trasy[0].przebieg), len(kopia.przebieg)
```

Out[54]:
```
(2, 3)
```

In [55]:
```
jan.ulubione_trasy
```

Out[55]:
```
[<Trasa nazwa='Z domu do pracy', przebieg=[Wspolrzedne(22.55, 51.24), Wspol
rzedne(22.57, 51.25)]>]
```

In [56]:
```
anna.ulubione_trasy
```

Out[56]:
```
[<Trasa nazwa='Spacer Anny', przebieg=[]>,
 <Trasa nazwa='Droga Anny do pracy', przebieg=[Wspolrzedne(22.6, 51.24), Ws
polrzedne(22.57, 51.25), Wspolrzedne(22.61, 51.26)]>]
```

Anna może teraz zmieniać skopiowaną trasę bez zmieniania trasy Jana. W tym przykładzie `deepcopy` utworzyło kopie zagnieżdżonych obiektów mutowalnych. Nie potrzebujemy osobnych kopii niemutowalnych napisów i liczb, żeby uzyskać tę niezależność.

Głębokie kopiowanie bez modułu `copy`

Dla znanej nam struktury możemy sami utworzyć wszystkie potrzebne obiekty. Poniższa funkcja tworzy nową listę tras. Dla każdej trasy tworzy nowy obiekt `Trasa`, a dla każdego punktu — nowy obiekt `Wspolrzedne`. Nie używa modułu `copy`.

In [57]:
```
def kopiuj_trasy_recznie(trasy):
    kopie = []
    for trasa in trasy:
        nowa_trasa = Trasa(trasa.nazwa)
        for punkt in trasa.przebieg:
            nowy_punkt = Wspolrzedne(punkt.dlugosc, punkt.szerokosc)
            nowa_trasa.dodaj_punkt(nowy_punkt)
        kopie.append(nowa_trasa)
    return kopie
```

Taką funkcją można zastąpić wywołanie `deepcopy` w metodzie `kopiuj_ulubione`:

`self.ulubione_trasy.extend(kopiuj_trasy_recznie(inny.ulubione_trasy))`
Wypróbujmy tę samą operację na nowym użytkowniku:

In [58]:
```
ola = Uzytkownik("Ola", "Lis")
```

In [59]:
```
ola.ulubione_trasy.extend(kopiuj_trasy_recznie(jan.ulubione_trasy))
```

In [60]:
```
ola.ulubione_trasy[0].przebieg[0].dlugosc = 22.70
```

In [61]:
```
ola.ulubione_trasy[0].przebieg[0]
```

Out[61]:
```
Wspolrzedne(22.7, 51.24)
```

In [62]:
```
jan.ulubione_trasy[0].przebieg[0]
```

Out[62]:
```
Wspolrzedne(22.55, 51.24)
```

Punkt oryginalnej trasy nie został zmieniony. Funkcja kopiuje wszystkie mutowalne poziomy tej konkretnej struktury, ale nie jest ogólnym zamiennikiem `deepcopy`.

Jeśli ten sam punkt występuje na dwóch trasach, pętla utworzy dwie osobne kopie. Nie zapamiętuje już skopiowanych obiektów. `deepcopy` zachowuje takie powiązania.

Wspólne punkty wewnątrz kopii

Jeśli dwie trasy Jana korzystają z tego samego punktu, `deepcopy` podczas jednego wywołania tworzy jedną kopię tego punktu. Obie skopiowane trasy będą ją współdzielić. To nadal osobny punkt względem oryginału Jana.

In [63]:
```
druga_trasa = Trasa("Powrót z pracy")
druga_trasa.dodaj_punkt(praca)
druga_trasa.dodaj_punkt(dom)
jan.dodaj_ulubiona(druga_trasa)
```

In [64]:
```
ewa = Uzytkownik("Ewa", "Lis")
ewa.kopiuj_ulubione(jan)
```

In [65]:
```
ewa.ulubione_trasy[0].przebieg[1] is ewa.ulubione_trasy[1].przebieg[0]
```

Out[65]: True

In [66]:
```
ewa.ulubione_trasy[0].przebieg[1] is praca
```

Out[66]: False

Osiągnęliśmy teraz całkowitą niezależność tras między użytkownikami.

To, które obiekty powinny być wspólne, wynika z konkretnych potrzeb programu.